Guide illustré

Virtualisation : ton calculateur dans un véhicule 3D (ROS 2 + Webots)

Projet Lighting System Control — Amina Borcheni — M2 IAA, ENET'Com — ROS 2 Lyrical, Webots R2025a, Ubuntu 26.04 (WSL 2)

Objectif : faire tourner **le même code C** que celui validé en SIL comme un vrai nœud ROS 2, connecté à une Tesla virtuelle dans Webots. La voiture mesure la lumière, sa vitesse et voit les véhicules avec sa caméra ; le calculateur décide ; les phares et clignotants de la voiture 3D obéissent. Un trajet automatique joue tous les cas du cahier des charges et donne un verdict : **17 / 17 PASS**.

Étape du cycle en V	Ce qui tourne	Environnement	Résultat
MIL unitaire + intégration	Modèles Simulink	Signaux de test	149 / 149 PASS
SIL	Code C généré (Embedded Coder)	Banc de test C	6 / 6 PASS (0 écart)
Virtualisation	**Le même code C, nœud ROS 2**	**Véhicule 3D Webots**	**17 / 17 PASS**
HIL (perspective)	Code C sur microcontrôleur	Banc temps réel	—



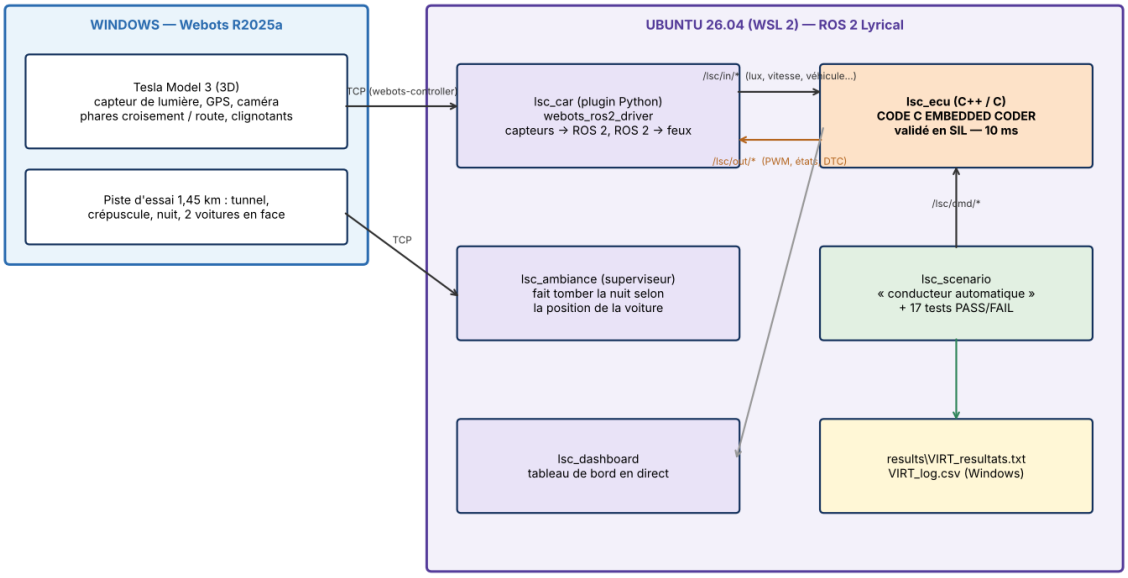
La Tesla virtuelle de nuit : feux de route allumés par l'AHB de ton calculateur (B3)

1. Pourquoi ROS 2 et Webots ?

Webots est un simulateur 3D de robots et de véhicules (physique, capteurs, caméra, lumières). **ROS 2** est le standard des logiciels robotiques et des prototypes de véhicules autonomes : des programmes (« nœuds ») échangent des messages sur des « topics ». Brancher le calculateur sur ROS 2 le rend utilisable avec n'importe quel simulateur ou véhicule réel qui parle ROS 2.

Pourquoi WSL ? ROS 2 est conçu d'abord pour Linux. Sous Windows, la méthode officielle est de faire tourner ROS 2 dans Ubuntu (WSL 2) et Webots sous Windows (il a besoin de la carte graphique) ; le paquet webots_ros2 relie les deux par TCP.

2. L'architecture



Les nœuds ROS 2 (Ubuntu) et la simulation Webots (Windows)

Nœud ROS 2	Langage	Rôle
lsc_ecu	C++ + code C	**Le calculateur** : appelle LSC_cmd() puis LSC_diag() toutes les 10 ms (code C du SIL, inchangé)
lsc_car	Python (plugin)	Lit les capteurs de la Tesla, publie /lsc/in/*, allume phares et clignotants selon /lsc/out/*, guide la voiture par GPS
lsc_ambiance	Python (superviseur)	Baisse le soleil, le ciel et le brouillard : la nuit tombe vraiment pour le capteur
lsc_scenario	Python	Conducteur automatique (commodo, clignotants, pannes, freinage, contact) + 17 vérifications
lsc_dashboard	Python	Affichage en direct : vitesse, lux, état des feux, AHB, DTC

Topic	Sens	Exemples
/lsc/in/*	Véhicule → calculateur	lux (capteur de lumière), speed (GPS), vehicle_detected (caméra), decel
/lsc/cmd/*	Conducteur → calculateur	light_switch, turn_left, flash, hazard, ignition, door_closed, cam_fault, lamp_fail
/lsc/out/*	Calculateur → véhicule	duty (8 PWM), light_state, ahb_state, hazard, ess, fmh, dtc

3. Installation (une seule fois)

1. Ubuntu dans WSL 2 (déjà présent : **Ubuntu 26.04**). Ouvrir « Ubuntu » depuis le menu Démarrer, ou **wsl -d Ubuntu** dans PowerShell.
2. Webots R2025a installé sous Windows (C:\Program Files\Webots).
3. Dans Ubuntu : dépôt officiel ROS 2, puis **sudo apt install ros-lyrical-desktop ros-dev-tools**.
4. webots_ros2 n'existe pas encore en paquet pour Lyrical : il est **compilé depuis les sources** (git clone, rosdep, colcon build).
5. Nos deux paquets **lsc_ecu** et **lsc_virtual** sont copiés dans ~/ros2_ws/src puis compilés avec colcon.

```
cd ~/ros2_ws && colcon build --packages-select lsc_ecu lsc_virtual
source install/local_setup.bash
```

4. Lancer la démonstration

Commande (terminal Ubuntu)	Effet
ros2 launch lsc_virtual lsc_launch.py	Trajet complet, sans tests automatiques (démonstration libre)
ros2 launch lsc_virtual lsc_launch.py scenario:=true	Trajet complet + conducteur automatique + 17 tests
ros2 run lsc_virtual dashboard	(2e terminal) tableau de bord en direct
ros2 topic pub --once /lsc/cmd/vehicle std_msgs/msg/Bool "{data: true}"	Exemple : simuler un véhicule en face à la main

Dans le calculateur, les seuils sont ceux du cahier des charges : rien n'a été ajusté pour la simulation. C'est le décor (soleil, tunnel) qui a été réglé pour produire des luminosités réalistes : ~31 000 lux au soleil, 150 lux dans le tunnel, ~25 lux la nuit.

5. Ce qu'on voit dans Webots



Jour : la Tesla sur la piste d'essai, feux de jour (DRL), 50 km/h



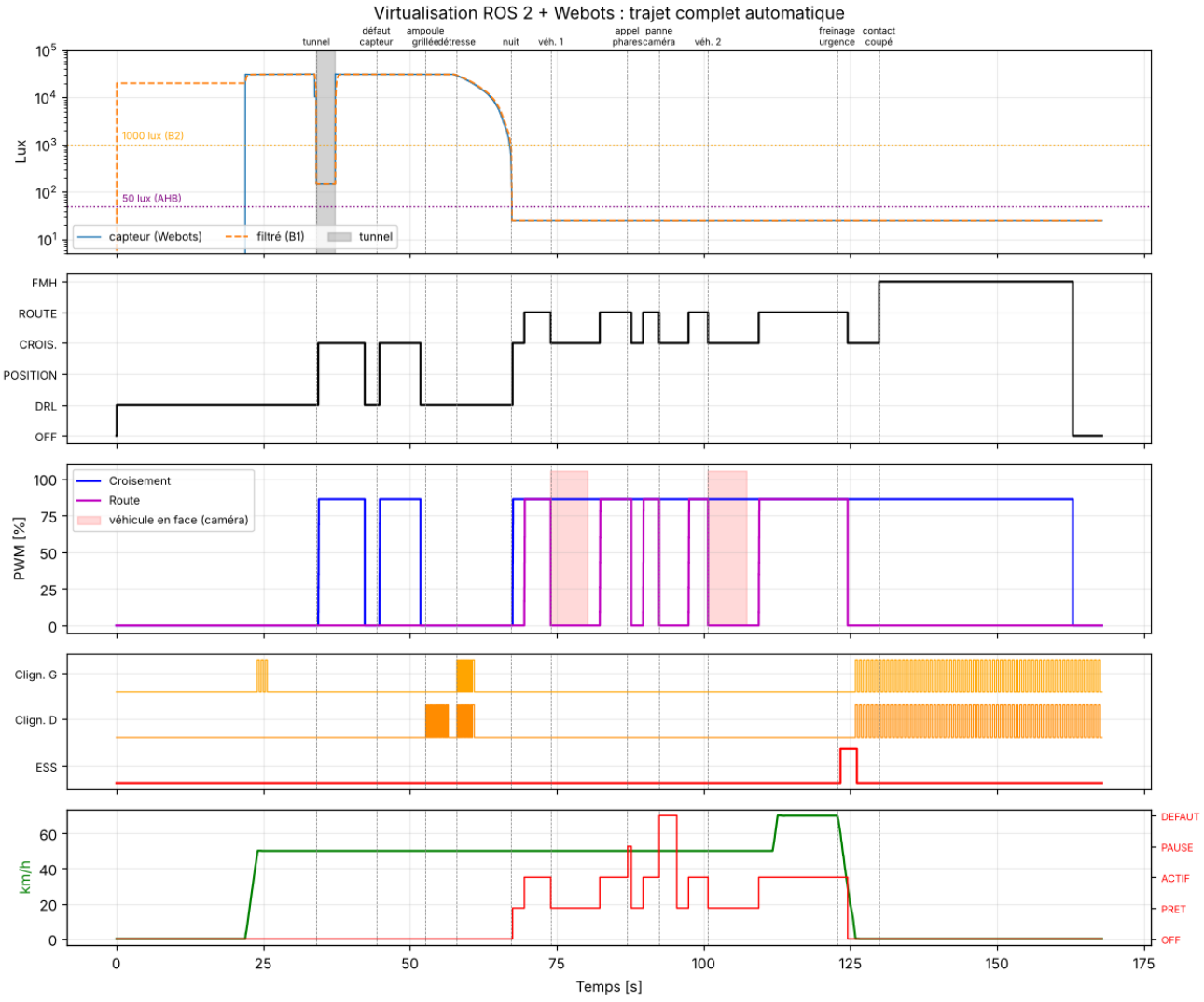
Nuit : feux de croisement (faisceau court) commandés par B2



Nuit : véhicule en face détecté par la caméra → feux de route coupés (B3, AHB en pause)

6. Le trajet automatique et ses 17 tests

Phase	Événement joué par lsc_scenario	Tests
Jour	Démarrage, impulsion courte du clignotant gauche	V-01, V-02
Jour	Tunnel de 40 m (150 lux)	V-04, V-05
Jour	Message du capteur de lumière absent 1,5 s	V-03
Jour	Ampoule du clignotant droit grillée + clignotant droit ; détresse	V-06, V-07, V-08
Crépuscule	Le soleil baisse (31 000 → 25 lux)	V-09
Nuit	Route libre à 50 km/h ; 2 voitures garées en face	V-10, V-11, V-12
Nuit	Appel de phares ; panne caméra 3 s	V-13, V-14
Nuit	70 km/h puis freinage d'urgence	V-15
Arrivée	Contact coupé de nuit, porte ouverte puis fermée	V-16, V-17



VIRT_trajet.png : luminosité, état des feux, PWM, clignotants/ESS, vitesse et état de l'AHB pendant tout le trajet

7. Résultats : 17 / 17 PASS

Test	Exigence	Vérification	Mesuré	Résultat
V-01	REQ-LSC-05	Jour, contact mis : DRL seuls	DRL	PASS
V-02	REQ-LSC-17	Impulsion courte : 3 clignotements	3	PASS
V-03	REQ-LSC-09	Message capteur absent > 0,5 s : croisement	0,52 s	PASS
V-04	REQ-LSC-08	Entrée de tunnel : croisement ≤ 0,4 s	0,31 s	PASS
V-05	REQ-LSC-07	Sortie de tunnel : DRL après 5 s	5,01 s	PASS
V-06	REQ-LSC-23	Ampoule grillée : hyper-clignotement	9 en 3 s	PASS
V-07	Diag B6	Défaut lampe mémorisé (DTC)	DTC = 8	PASS
V-08	B5	Commande détresse	active	PASS
V-09	REQ-LSC-06	Crépuscule : croisement automatique	932 lux	PASS
V-10	REQ-LSC-10/11	Nuit, > 40 km/h, route libre : route AHB	4,5 s	PASS
V-11	REQ-LSC-13	Véhicule en face : route coupée	0,01 s	PASS
V-12	REQ-LSC-12	Route libre : route rallumée après 2 s	2,01 s	PASS
V-13	REQ-LSC-15	Appel de phares : route ON, AHB en pause	PAUSE	PASS
V-14	REQ-LSC-14	Panne caméra : AHB en défaut	0,02 s	PASS
V-15	REQ-LSC-20	Freinage d'urgence : feux stop clignotants	ESS actif	PASS
V-16	REQ-LSC-25	Contact coupé de nuit : follow-me-home	FMH	PASS
V-17	REQ-LSC-25	Extinction 30 s après la porte	30,03 s	PASS

Fichiers produits automatiquement : **results\VIRT_resultats.txt** (verdicts + horodatage des événements) et **results\VIRT_log.csv** (toutes les entrées/sorties, 50 Hz). Les temps mesurés sont proches de ceux du MIL (tunnel 0,31 s, extinction 5,01 s, rallumage AHB 2,01 s) : **le comportement est le même du modèle au véhicule virtuel**.

8. Les problèmes rencontrés (et ce qu'ils nous ont appris)

La virtualisation a demandé plusieurs itérations. Chaque problème venait de l'**environnement de simulation**, jamais du calculateur — mais il fallait le prouver à chaque fois avec les mesures.

Problème observé	Cause	Correction
La Tesla sort de la route dans les virages	Le suivi de voie de la démo détecte la ligne jaune à la caméra ; perturbé par nos phares et le tunnel	Piste d'essai rectiligne + guidage par GPS (comme une piste constructeur)
Vitesse affichée 0 km/h	getCurrentSpeed() de Webots invalide dans ce mode	Vitesse mesurée par le GPS
Pas de feux de route la nuit	La lumière ambiante du soleil restait mesurée : ~10 000 lux « la nuit »	Soleil remplacé par une lumière dont on règle directe + ambiante → 25 lux
Clignotants invisibles	wb_driver_set_indicator absent de la bibliothèque installée	LED orange pilotées directement par les PWM de B5
Simulation bloquée à 0 s	Le superviseur d'ambiance n'était pas lancé ; Webots l'attendait	Lancement systématique du superviseur
V-01 en échec (16/17)	Le test commençait avant la découverte ROS 2 (2,9 s)	Vérification à partir du 1er message reçu du calculateur

Message pour le jury : dans un banc de simulation, il faut toujours séparer les défauts du système testé et ceux du banc. Ici, le journal (VIRT_log.csv) a permis de montrer que le calculateur avait raison à chaque fois : par exemple, à 10 000 lux il ne doit pas allumer les feux de route.

Limites

Limite	Explication
Pas temps réel strict	Le nœud tourne avec un timer de 10 ms sous Linux non temps réel ; Webots tourne à ~0,9× le temps réel
Capteurs idéalisés	Le capteur de lumière ne mesure que les sources de lumière de Webots ; l'éclairage du tunnel est fixé à 150 lux
Détection simplifiée	La caméra utilise la « recognition » de Webots (objets annotés), pas un réseau de neurones
Pas de bus CAN	Les signaux passent par des topics ROS 2 ; un vrai véhicule utiliserait CAN / AUTOSAR COM

9. Questions probables du jury

- **Est-ce le même code que le SIL ?**
  Oui : les fichiers C générés par Embedded Coder sont copiés tels quels dans le paquet lsc_ecu ; seule l'interface ROS 2 (lecture des topics, appel de LSC_cmd et LSC_diag) est ajoutée autour.
- **Pourquoi ROS 2 plutôt que Simulink seul ?**
  ROS 2 est le standard des prototypes de véhicules et robots ; il permet de brancher le même calculateur sur un simulateur 3D, un autre logiciel ADAS ou un vrai véhicule.
- **Comment la caméra détecte-t-elle les véhicules ?**
  Par la fonction « recognition » de Webots, qui donne les objets visibles et leur distance ; un objet de type véhicule à moins de 90 m met vehicle_detected à 1.

- **Les résultats sont-ils reproductibles ?**
  Oui : le scénario est déclenché par la position de la voiture, et deux trajets successifs ont donné les mêmes mesures (tunnel 0,30–0,31 s, AHB 2,0 s, FMH 30,0 s).
- **Quelle est la suite ?**
  HIL : le même code C sur une carte STM32 avec de vraies LED, reliée au simulateur ; puis génération AUTOSAR (ARXML, RTE) avec Simulink Coder.

Bilan des tests du projet	Tests	Résultat
MIL unitaire (B1 à B8)	113	113 PASS
MIL intégration (4 scénarios)	36	36 PASS
SIL (code C)	6	6 PASS
Virtualisation ROS 2 + Webots	17	17 PASS
Total	**172**	**172 PASS**